

## TD 4 — Comprendre le droit du logiciel libre — CORRIGÉ

### Objectifs

- Comprendre les implications pratiques du droit d’auteur pour un développeur
  - Identifier qui détient les droits selon le statut (salarié, stagiaire, freelance)
  - Analyser un cas juridique emblématique
  - Réfléchir aux enjeux des brevets logiciels et de l’interopérabilité
- 

### Exercice 1 — Qui détient les droits sur votre code ? (25 min)

#### Scénarios

##### Scénario A :

Marie est développeuse salariée chez TechCorp. Dans le cadre de son travail, elle développe une application de gestion de stock pendant ses heures de bureau, sur son ordinateur professionnel.

→ Qui détient les droits patrimoniaux ? **TechCorp (l’employeur)**

→ Quel article du CPI s’applique ? **Article L.113-9 du Code de la Propriété Intellectuelle**

**Explication :** L’article L.113-9 CPI prévoit une cession automatique des droits patrimoniaux à l’employeur pour les logiciels créés par un salarié dans l’exercice de ses fonctions ou d’après les instructions de l’employeur. Les droits moraux restent à Marie, mais les droits d’exploitation appartiennent à TechCorp.

---

##### Scénario B :

Le week-end, Marie développe chez elle une application de gestion de photos personnelles, sans rapport avec son travail. Elle utilise son ordinateur personnel et ses propres outils.

→ Qui détient les droits ? **Marie**

→ Pourquoi ? **Le développement est hors du cadre du contrat de travail : temps personnel, matériel personnel, sujet sans rapport avec l’activité professionnelle. L’article L.113-9 ne s’applique pas.**

**Explication :** Les trois critères (temps, matériel, sujet) pointent vers un projet personnel. Marie conserve tous ses droits. Toutefois, il est prudent de vérifier son contrat de travail qui pourrait contenir des clauses spécifiques.

---

##### Scénario C :

Marie a une idée d’outil open source pendant ses vacances. Elle le développe chez elle le soir, mais utilise parfois son ordinateur professionnel pour tester. L’outil n’a aucun rapport avec l’activité de TechCorp.

→ Qui détient les droits ? **Situation ambiguë — probablement Marie, mais risque de contestation**

→ Quels risques pour Marie ?

- L’utilisation du matériel professionnel crée une zone grise
- TechCorp pourrait revendiquer des droits
- Risque de litige coûteux
- Conseil : obtenir un accord écrit de l’employeur ou utiliser uniquement du matériel personnel

**Explication** : Même si le sujet n'a aucun rapport avec l'activité de l'entreprise, l'utilisation des ressources de l'employeur peut créer une ambiguïté juridique. Certains contrats de travail prévoient que tout code écrit avec les outils de l'entreprise lui appartient.

---

#### Scénario D :

Paul est étudiant et fait un stage rémunéré de 6 mois chez StartupXYZ. Pendant son stage, il développe un module important pour leur produit principal. La convention de stage ne mentionne pas la propriété intellectuelle.

→ Qui détient les droits selon la loi française ? **Paul (le stagiaire)**

→ Pourquoi cette situation est-elle différente d'un salarié ? **L'article L.113-9 CPI ne mentionne que les « salariés » et « agents publics ». Un stagiaire n'est pas un salarié au sens juridique, même s'il est rémunéré (gratification). Le stage n'est pas un contrat de travail.**

→ Que devrait prévoir StartupXYZ pour sécuriser la situation ?

- Une clause de cession de droits dans la convention de stage
- Ou un contrat séparé de cession de droits patrimoniaux
- Important : la cession doit être explicite et détaillée (types de droits, durée, territoire, supports)

**Explication** : C'est un piège classique pour les startups. Sans cession explicite, Paul pourrait théoriquement interdire à StartupXYZ d'utiliser le code qu'il a écrit, ou demander une rémunération supplémentaire.

---

#### Scénario E :

Sophie est freelance. Elle développe un site web pour un client. Le contrat ne mentionne pas la propriété intellectuelle.

→ Qui détient les droits en France par défaut ? **Sophie (la freelance/auteure)**

→ Pourquoi est-ce un problème pour le client ?

- Le client n'a que le droit d'utiliser le site, pas de le modifier
- Il ne peut pas faire évoluer le site par un autre prestataire
- Il ne peut pas revendre le code
- Sophie pourrait théoriquement lui retirer le droit d'utilisation

→ Que devrait prévoir le contrat ?

- Une clause de cession des droits patrimoniaux
- Détaillant : droits cédés, durée, territoire, supports
- Avec une rémunération spécifique pour la cession (obligatoire en France)

**Explication** : En France, l'auteur conserve ses droits par défaut. Contrairement au régime américain du "work for hire", le fait de payer pour un travail ne transfère pas automatiquement les droits.

---

#### Scénario F :

Lucas contribue bénévolement à un projet open source sous licence MIT pendant son temps libre. Il est salarié par ailleurs.

→ Qui détient les droits sur ses contributions ? **Lucas**

→ Ces contributions peuvent-elles poser problème avec son employeur ?

- Potentiellement oui, si :

- Le projet est en rapport avec l'activité de l'employeur
- Lucas utilise des connaissances confidentielles acquises au travail
- Le contrat de travail contient une clause de non-concurrence ou de PI étendue
- **En pratique : vérifier le contrat de travail et, en cas de doute, informer l'employeur**

**Explication :** La licence MIT permet à Lucas de contribuer librement. Ses contributions sont sous MIT, donc son employeur (et tout le monde) peut les utiliser. Le risque est surtout contractuel (clauses du contrat de travail).

---

## Exercice 2 — Oracle vs Google : l'affaire des APIs Java (20 min)

### Questions et réponses

#### 1. Pourquoi Oracle considérerait-il que les APIs Java devaient être protégées par le droit d'auteur ?

Oracle argumentait que :

- Les déclarations d'APIs représentent un **travail créatif original** (choix des noms, organisation, structure)
  - La **structure, séquence et organisation** (SSO) des APIs est une expression protégeable
  - Google aurait pu créer ses propres APIs différentes
  - La copie de 11 500 lignes représente une **reproduction substantielle**
  - Sans protection, l'investissement de Sun/Oracle dans Java ne serait pas récompensé
- 

#### 2. Quels arguments Google a-t-il avancés pour sa défense ?

Google a argumenté que :

- Les APIs sont **fonctionnelles** et nécessaires pour l'interopérabilité
  - Seules les **déclarations** ont été copiées, pas l'implémentation (réécrite entièrement)
  - Les développeurs avaient besoin de retrouver les mêmes noms de méthodes pour utiliser leurs compétences Java
  - Protéger les APIs créerait un **monopole sur un langage**
  - L'usage était **transformatif** : Android est une plateforme différente de Java SE
  - C'est un cas de **fair use** (usage équitable)
- 

#### 3. Qu'est-ce que le "fair use" (usage équitable) en droit américain ? Pourquoi la Cour Suprême l'a-t-elle appliqué ici ?

##### Définition du fair use :

Exception au copyright américain permettant l'utilisation sans autorisation dans certains cas. Les tribunaux examinent 4 facteurs :

1. **But et caractère de l'utilisation** (commercial vs éducatif, transformatif ?)
2. **Nature de l'œuvre copiée** (créative vs factuelle)
3. **Quantité copiée** par rapport à l'ensemble
4. **Effet sur le marché** de l'œuvre originale

##### Application par la Cour Suprême (2021) :

- L'usage était **transformatif** : Android est un environnement mobile, différent de Java SE
- Google n'a copié que ce qui était **nécessaire** pour permettre aux développeurs d'utiliser leurs compétences

- Les APIs sont plus **fonctionnelles** que créatives
- Oracle n'avait pas réussi sur le marché mobile, donc peu d'effet sur son marché

**Note importante :** La Cour n'a PAS tranché sur la question de savoir si les APIs sont protégeables par copyright. Elle a supposé qu'elles l'étaient et a jugé que l'usage était fair use quand même.

---

#### 4. Si Oracle avait gagné, quelles auraient été les conséquences ?

##### Pour les développeurs :

- Risque juridique en réimplémentant des APIs existantes
- Impossibilité de créer des alternatives compatibles
- Dépendance accrue envers les éditeurs de langages/platformes

##### Pour l'interopérabilité :

- Mort des projets comme Wine, Mono, ReactOS
- Impossibilité de créer des implémentations alternatives
- Marchés captifs renforcés

##### Pour les langages de programmation :

- Les créateurs de langages pourraient "verrouiller" leur écosystème
  - Risque pour les langages open source implémentant des APIs standard
  - Fragmentation accrue
- 

#### 5. Cette affaire aurait-elle eu la même issue en Europe ? Pourquoi ?

##### Probablement une issue similaire, mais pour des raisons différentes :

- La **directive 91/250/CE** (aujourd'hui 2009/24/CE) prévoit explicitement que les **interfaces nécessaires à l'interopérabilité** ne sont pas protégées
- L'article 6 autorise la **décompilation pour interopérabilité**
- Le droit européen protège explicitement l'interopérabilité comme objectif légitime

**Différence clé :** En Europe, la question aurait été tranchée sur le fond (les APIs d'interopérabilité ne sont pas protégeables), pas sur une exception (fair use). Le résultat pratique serait similaire, mais la base juridique plus claire.

**Note :** Le fair use n'existe pas en droit français/européen. Il y a des exceptions limitées (citation, parodie, etc.) mais pas d'analyse au cas par cas comme aux US.

---

### Exercice 3 — Interopérabilité et reverse engineering (15 min)

#### Questions et réponses

##### 1. Ces projets ont-ils eu besoin de "décompiler" du code Microsoft pour fonctionner ?

###### Wine :

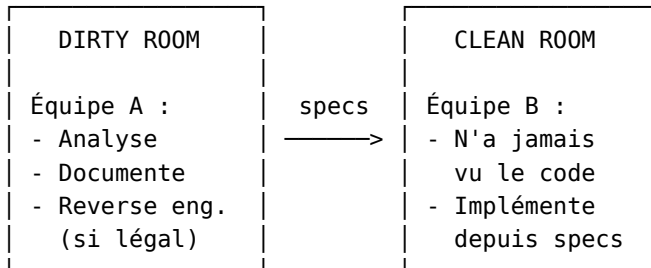
- Non, Wine n'a pas décompilé le code Windows
- Wine a documenté le comportement des APIs Windows par **observation** (tests, documentation publique, messages d'erreur)
- Technique de **clean room** utilisée pour éviter toute contamination

###### Samba :

- Situation plus complexe
  - Initialement, reverse engineering du protocole SMB par observation du trafic réseau
  - Microsoft a été contraint par l'UE (décision antitrust 2004) de **documenter ses protocoles**
  - Depuis, Samba utilise cette documentation officielle
- 

## 2. Comment la technique “clean room” a-t-elle été utilisée pour ces projets ?

**Principe du clean room :**



**Application :**

- **Équipe A** (dirty room) : étudie le comportement de Windows/SMB, documente les interfaces, les formats, les protocoles. Cette équipe ne code PAS.
- **Équipe B** (clean room) : reçoit uniquement la documentation technique, sans jamais voir le code original. Elle implémente une version propre.

**Pourquoi ça marche juridiquement :**

- L'équipe B ne peut pas avoir copié (consciemment ou non) du code qu'elle n'a jamais vu
  - Seules les **idées et interfaces** sont transmises, pas l'expression (le code)
  - Les idées ne sont pas protégées par le droit d'auteur
- 

## 3. En Europe, la décompilation pour interopérabilité est autorisée sous certaines conditions. Quelles sont ces conditions ?

**Directive 91/250/CE, Article 6 – Conditions cumulatives :**

1. **Nécessité** : Les informations d'interface ne sont pas facilement accessibles autrement
2. **Légitimité** : L'acte est accompli par le licencié ou une personne autorisée à utiliser le programme
3. **Limitation** : Seules les parties du programme nécessaires à l'interopérabilité peuvent être décompilées
4. **Finalité** : L'objectif est uniquement d'obtenir les informations nécessaires à l'interopérabilité avec un programme créé de façon indépendante

**Restrictions supplémentaires :**

- Les informations obtenues ne peuvent pas être utilisées à d'autres fins
  - Elles ne peuvent pas être communiquées à des tiers (sauf si nécessaire pour l'interopérabilité)
  - Elles ne peuvent pas servir à créer un programme “substantiellement similaire”
- 

## 4. Pourquoi Microsoft ne peut-il pas empêcher juridiquement l'existence de Samba en Europe ?

**Plusieurs raisons :**

1. **Directive sur l'interopérabilité** : Le reverse engineering pour interopérabilité est un droit en Europe. Microsoft ne peut pas l'interdire contractuellement.
2. **Décision antitrust de la Commission européenne (2004)** :
  - Microsoft a été condamné pour abus de position dominante
  - Obligation de documenter les protocoles de ses serveurs
  - Amende de 497 millions d'euros (puis 899 millions supplémentaires pour non-conformité)
  - Le Protocol Freedom Information permet aux concurrents d'implémenter les protocoles
3. **Les protocoles réseau ne sont pas protégeables** en tant que tels (ce sont des spécifications fonctionnelles)
4. **Samba n'a pas copié de code Microsoft** — c'est une implémentation indépendante

**Ironie** : Microsoft est aujourd'hui contributeur à Samba et utilise des composants open source dans Windows.

---

## Exercice 4 — Débat : Pour ou contre les brevets logiciels ? (15 min)

Exemples et contre-exemples pour chaque argument

### A. "Les brevets protègent l'investissement en R&D des entreprises innovantes."

- **Exemple favorable** : Qualcomm finance sa R&D sur les technologies mobiles (4G, 5G) grâce aux royalties de ses brevets
  - **Contre-exemple** : Les startups logicielles innoveraient souvent sans brevets (open source, SaaS). Les brevets sont coûteux à déposer et à défendre.
- 

### B. "Sans brevets, les grandes entreprises copieraient les innovations des startups."

- **Exemple favorable** : Le brevet peut dissuader la copie et faciliter les levées de fonds (actif valorisable)
  - **Contre-exemple** : En pratique, les grandes entreprises peuvent contourner les brevets ou racheter les startups. Les petites entreprises n'ont souvent pas les moyens de poursuivre les géants.
- 

### C. "Les brevets encouragent la divulgation des innovations (vs le secret)."

- **Exemple favorable** : Un brevet publié permet à d'autres de s'en inspirer après expiration (20 ans)
  - **Contre-exemple** : Pour le logiciel, le code source n'est généralement pas dans le brevet. Le secret commercial (code fermé) reste possible sans brevet.
- 

### D. "L'industrie du logiciel a innové pendant 50 ans sans brevets."

- **Exemple** : Les innovations fondamentales (TCP/IP, HTTP, HTML, compilateurs, systèmes d'exploitation) ont été créées sans brevets logiciels significatifs. Linux, Apache, le Web sont nés hors du système de brevets.
  - **Nuance** : Les brevets logiciels se sont développés surtout à partir des années 1990 aux US.
- 

### E. "Les brevets logiciels sont souvent trop larges et vagues."

- **Exemple** : Le brevet Amazon "1-Click" (achat en un clic) — concept évident breveté
- **Exemple** : Brevets sur les "barres de progression", les "paniers d'achat en ligne"
- **Raison** : Les examinateurs de brevets manquent souvent d'expertise technique en logiciel, et le concept "d'état de l'art" est mal défini

---

#### F. “Les ‘patent trolls’ utilisent les brevets pour extorquer de l’argent, pas pour innover.”

- **Exemple** : L’affaire Blackberry/NTP — NTP n’avait aucun produit, mais a obtenu 612,5 M\$ de RIM (Blackberry)
  - **Exemple** : Intellectual Ventures, qui détient des milliers de brevets sans produire de logiciel
  - **Modèle économique** : Acheter des brevets, menacer de procès, obtenir des règlements
- 

#### G. “Les petites entreprises n’ont pas les moyens de se défendre contre les litiges.”

- **Exemple** : Un procès en brevet coûte en moyenne 2-5 millions de dollars aux US
  - **Conséquence** : Les startups préfèrent souvent payer des licences (même injustifiées) plutôt que de risquer un procès
  - **Asymétrie** : Les grandes entreprises accumulent des brevets pour négocier des “cross-licenses”
- 

### Questions de synthèse

#### 2. Pourquoi les brevets logiciels sont-ils exclus “en tant que tels” en Europe mais acceptés aux États-Unis ?

##### Europe (Convention de Munich, Article 52) :

- Les “programmes d’ordinateurs en tant que tels” sont exclus
- Raison historique : le logiciel était vu comme une expression mathématique/abstraite
- **Mais** : Les “inventions mises en œuvre par ordinateur” peuvent être brevetées si elles apportent un “effet technique”
- En pratique : zone grise, l’OEB accorde des brevets sur certains logiciels

##### États-Unis :

- Pas d’exclusion explicite dans la loi
  - Décisions jurisprudentielles favorables (Diamond v. Diehr, 1981)
  - Culture plus favorable aux brevets comme outil commercial
  - **Évolution récente** : Alice v. CLS Bank (2014) a restreint certains brevets logiciels “abstraits”
- 

#### 3. Quelle position l’écosystème open source adopte-t-il généralement face aux brevets ?

- **Opposition historique** aux brevets logiciels (FSF, OSI, EFF)
  - **Licences défensives** : Apache 2.0 et GPL v3 incluent des clauses de grant/rétorsion de brevets
  - **Pools défensifs** : Open Invention Network (OIN) — pool de brevets pour protéger Linux
  - **Patent pledges** : Promesses de non-agression (Red Hat, Google, Microsoft via OIN)
  - **Lobbying** contre les brevets logiciels en Europe (victoire 2005)
- 

#### 4. Certaines entreprises ouvrent leurs brevets. Pourquoi ?

**Tesla (2014)** : A ouvert ses brevets sur les véhicules électriques

- **Raison** : Accélérer l’adoption des véhicules électriques pour agrandir le marché
- Tesla était petit face aux constructeurs traditionnels, avait intérêt à un écosystème large

**IBM** : A donné 500 brevets à l’open source

- **Raison** : Positionnement stratégique pro-Linux, relations publiques, réduction des risques de litiges

**Google (OIN)** :

- **Raison** : Protection défensive de l'écosystème Android/Linux

#### Motivations communes :

- Créer un écosystème / standard de facto
- Améliorer l'image de marque
- Réduire les risques de litiges réciproques
- Accélérer l'innovation dans un domaine où l'entreprise n'est pas dominante

## Exercice 5 — Les marques dans l'open source (si temps restant)

### Questions et réponses

#### 1. Pourquoi Debian a-t-il dû renommer Firefox ?

##### Contexte :

- Mozilla possède les marques "Firefox" et le logo du renard
- Pour utiliser ces marques, il faut distribuer Firefox **sans modification** ou obtenir l'accord de Mozilla
- Debian applique des patches de sécurité et modifie certains paramètres (moteur de recherche par défaut, etc.)
- Mozilla n'acceptait pas que ces versions modifiées portent le nom Firefox

##### Résultat :

- Debian a créé "Iceweasel" (code identique, nom et logo différents)
- Utilisé de 2006 à 2016
- Résolu en 2016 quand Mozilla a assoupli sa politique et Debian a accepté certaines conditions

#### 2. Le code de Firefox est-il libre ? Le nom et le logo sont-ils libres ?

| Élément       | Libre ?              | Protection                      |
|---------------|----------------------|---------------------------------|
| Code source   | <b>Oui</b> (MPL 2.0) | Droit d'auteur                  |
| Nom "Firefox" | <b>Non</b>           | Marque déposée                  |
| Logo (renard) | <b>Non</b>           | Marque déposée + droit d'auteur |

**Conséquence** : On peut forker le code librement, mais pas utiliser le nom/logo sans autorisation.

#### 3. Autres exemples de logiciels libres avec des marques protégées

| Projet                    | Marque protégée par | Exemple de fork renommé                  |
|---------------------------|---------------------|------------------------------------------|
| <b>Red Hat</b>            | IBM/Red Hat         | CentOS, Rocky Linux, AlmaLinux           |
| <b>Docker</b>             | Docker Inc.         | Podman (alternative, pas fork)           |
| <b>Linux</b>              | Linux Foundation    | — (peu de conflits)                      |
| <b>Ubuntu</b>             | Canonical           | Linux Mint (dérivé)                      |
| <b>Android</b>            | Google              | LineageOS, /e/OS                         |
| <b>Chromium</b> vs Chrome | Google              | Chromium = libre, Chrome = marque Google |
| <b>Java</b>               | Oracle              | OpenJDK (nom différent)                  |

#### 4. Une marque protégée est-elle compatible avec les principes du logiciel libre ?

##### Arguments pour la compatibilité :

- Les 4 libertés (utiliser, étudier, modifier, redistribuer) concernent le **code**, pas le nom
- La marque protège les **utilisateurs** contre la confusion (qualité, origine)
- On peut toujours forker et renommer
- Les fondations (Mozilla, Linux Foundation) utilisent les marques pour financer le projet

##### Arguments pour l'incompatibilité :

- Friction pour les distributions (Debian/Iceweasel)
- Peut décourager les forks et la diversité
- Asymétrie de pouvoir entre le détenteur et la communauté
- Vision "puriste" : tout devrait être libre, y compris le nom

##### Position pragmatique (majoritaire) :

- Les marques sont acceptables si elles ne restreignent pas les libertés fondamentales
  - La clé est la **politique de marque** : Mozilla a fini par assouplir la sienne
  - Debian a des règles strictes (DFSG) mais accepte les marques raisonnables
- 

## Synthèse du TD

### Points clés à retenir

#### 1. Droits d'auteur et statut :

- Salarié → cession automatique à l'employeur (L.113-9 CPI)
- Stagiaire → conserve ses droits (attention !)
- Freelance → conserve ses droits par défaut

#### 2. APIs et interopérabilité :

- Oracle vs Google : fair use, mais question non tranchée sur le fond
- Europe : protection explicite de l'interopérabilité (directive 91/250)

#### 3. Brevets logiciels :

- Acceptés aux US, exclus "en tant que tels" en Europe
- Écosystème open source généralement opposé
- Solutions défensives : OIN, clauses dans les licences

#### 4. Marques :

- Distinctes du code (peuvent être protégées même si le code est libre)
- Permettent de protéger l'identité du projet
- Fork = OK, mais avec un nouveau nom